SHIVANSHU CHAUHAN

Technical Artist | Rendering, shaders and optimisation | Unreal Engine 5, Unity

Portuguese passport (EU citizen), no visa sponsorship needed | Open to relocation or remote

Showreel: **youtube.com/@shivanshuchauhan3032** | Portfolio: **shadertrix.github.io**
LinkedIn: **linkedin.com/in/shivanshu-chauhan-89095b186** | Phone: **+91 84248 27266**
ArtStation: **artstation.com/shivanshuchauhan** | Email: **shivanshuchauhan1998@gmail.com**

PROFILE

Technical artist, five years in mobile games. At Gameberry Labs I own the frame budget on two live titles and build the shaders, VFX and editor tools that go into them.

Most of my own work for the last year has been in Unreal 5: a Lumen scene I took from 22 fps to 43, a data-driven PCG forest tool, a deformable sand system, a Control Rig creature, Niagara effects and a handful of editor scripts. Performance is the part I'm best at.

EXPERIENCE

Gameberry Labs — Bengaluru
July 2024 – Present

Technical Artist II | Sorry! World by Hasbro + Backgammon Friends Online

- Keep two live titles at 60 fps on low and mid-range Android and iOS. I set the art budgets and find the overdraw, state changes and draw call spikes before they ship.
- Built three rendering systems that are live in Sorry! World: quadtree occlusion culling, SDF text instancing and a tiled image shader. All three cut draw calls on the heaviest screens. Details below.
- Rebuilt the particle and destruction VFX on the GPU using compute shaders. CPU instancing had been the limit on how many effects we could run at once.
- Wrote the Unity editor tools the art team uses day to day: batch validators, atlas auditors, debug overlays, custom inspectors.

Dirtcube Interactive — Mumbai
March 2021 – July 2024

3D Artist | GameStarz

- Modelled, textured and optimised real-time assets for Unity on tight poly and texture budgets, and set up the PBR lighting for Android and iOS.
- Rigged and animated characters in Maya and Blender. Ran the 2D character pipeline in Spine2D.

SHIPPED TITLES

Sorry! World (Hasbro)
Gameberry Labs / Hasbro
Technical Artist II

Backgammon Friends Online
Gameberry Labs
Associate Technical Artist

GameStarz
Dirtcube Interactive
3D Artist

CORE SKILLS

Unreal Engine 5: Material Editor | Niagara | PCG | Control Rig | Nanite | Lumen | Virtual Shadow Maps | Blueprints | C++ | Python editor scripting | Editor Utility Widgets

Unity: Built-in / URP / HDRP | C# | Jobs & Burst | Custom editor tools and inspectors | Mobile performance budgeting

Rendering & GPU: HLSL | GLSL | Compute shaders | SDF rendering | Render targets | Draw-call and overdraw reduction | Scalability tiers | LOD and culling

Profiling: Unreal Insights | ProfileGPU and stat commands | Unity Profiler and Frame Debugger | CPU/GPU budgeting

Pipeline: Git | Python | Asset validation and batch tooling

DCC: Blender | Maya | Substance Designer & Painter | ZBrush | Photoshop | Spine2D

SELECTED PROJECTS — UNREAL ENGINE 5

All six are in the showreel on shadertrix.github.io. Numbers measured on an M3 Pro laptop, macOS/Metal, SM6 renderer.

Scene Optimisation Case Study *Lumen sci-fi environment*

- Took a Lumen scene from 22 fps to 43. GPU time went from 43.9 ms to 22.2 ms at Epic, and the worst frames from 61.8 ms to 39.9 ms.
- Built Low, Medium and High tiers with a runtime preset switcher. High went from 37 to 64.5 fps.
- Profiled with Unreal Insights, ProfileGPU and stat commands, then worked through lighting, shadows, materials and content. Written up as a case study with the per-pass numbers behind each change.

Procedural Forest Tool *Built in PCG*

- One PCG graph, driven entirely by data assets. Species, density, spacing, scale and terrain rules all live in the data, so a new biome takes no graph edits.
- Rivers and paths run off splines, clearing foliage and placing props as you move them.
- Custom editor panel puts every layer in one place. The showcase scene generates about 11,400 instances, 688 of them trees.

Interactive Sand System *Materials and render targets*

- Footprints and object tracks write into an R16 render target and fade over time. No per-frame CPU cost.
- Nanite tessellation and runtime displacement drive the surface. Custom HLSL handles the capture, the blending and the decay.

Mecha Scorpion *Control Rig creature*

- Full body procedural IK. Legs line-trace to the terrain on a single channel and the body follows.
- Tail lag and weapon recoil come from sim nodes. It's playable with movement, aim and fire, so the rig gets tested under real input.

Traversal VFX *Niagara | Inspired by inFAMOUS: Second Son*

- Smoke and neon traversal powers in Niagara. GPU particles, ribbons, trails and emissive light, switchable at runtime.

Editor Tools *C++ and Slate*

- **Level Mesh Instancer** finds duplicate static meshes in a level and converts them to ISM/HISM. Shows the counts and which actors change before you commit.
- **Material Instance Editor** opens a material's whole instance family as a spreadsheet: one row per instance, one column per parameter, every cell editable in place.
- **VAT baker** bakes vertex animation textures.

SELECTED SYSTEMS — UNITY

Shipped in Sorry! World *Production | Hasbro*

- **Quadtree occlusion culling.** Culls fully hidden on-screen elements each frame and drops their draw calls. Burst jobs keep it cheap on big canvases. No false culls in production.
- **SDF text instancing.** Bakes per-instance style (colour, outline, underlay, dilate) into mesh UVs. Lots of differently styled text then shares one material and batches into a single draw call.
- **Tiled image shader.** Large artwork gets sliced into atlas-sized tiles and reassembled on one quad. Cell lookup and UV remap are shader-side, so atlas size limits stop mattering and it stays one draw call.

Personal Projects *Unity | Compute Shaders*

- **GPU mass-spring soft body and cloth.** Structural, shear and bend springs solved in parallel in a compute shader with RK4 integration, plus collisions, wind, pinning and grab interaction.
- **Screen-space character voxelisation.** Animated characters rebuilt as dynamic voxels in URP using custom MRT render passes and compute shaders.
- **GPU jump flood.** Compute-shader Jump Flood Algorithm that turns any texture into a signed distance field and nearest-seed map, used for glow, outline and SDF effects.
- **GPU flow-field crowd.** Thousands of agents follow a CPU-baked flow field and resolve their own collisions against a GPU-side BVH of the level.

EDUCATION

KJ Somaiya College of Science & Commerce — Mumbai 2020
BSc Computer Science

MAAC (Maya Academy of Advanced Cinematics) — Mumbai 2018 – 2020
Game Art and Development